

**CPC Search**

CPC search

Go

About Us

Our Mission
Who We Are

Contact Us

CPC Information
CPC Web Team

[HOME](#) > [Monitoring and Data](#) > [Oceanic & Atmospheric Data](#) > Meteorological Data Servers

Fast Downloading of Grib, Part 2

NEWS

Jan 2, 2019: nomads.ncep.noaa.gov is changing the URLs from http:// to https://. A version of get_gfs.pl with the new URLs was released 12/31/2018. You may need to get a newer version of cURL if you have problems.

Wrappers @ NCDC

While the procedure detailed in part 1 is straight forward, it could be easier. I don't like looking for and typing out URLs. Writing loops takes time too. Less experienced people like it even less. Dan Swank wrote a nice wrapper to download data for the North American Regional Reanalysis (NARR). It worked so well that he did [get-httpssubset.pl](#). During May 2006, 95% of the NCDC-NOMADS downloads were done using cURL.

Wrappers @ NCEP (NOMADS): get_gfs.pl

At NCEP, we wanted people to (1) get forecasts using partial-http transfers rather than ftp2u and (2) move off the nomad servers to the more reliable NCO servers. So get_gfs.pl was born. Wanted the script to be easy to use, easy to reconfigure, easier to install and work with Windows.

Requirements

1. [get_gfs.pl](#).
2. perl
3. [cURL](#)

Configuration

1. The cURL executable needs to be downloaded and put in a directory on your \$PATH.
2. The first line of get_gfs.pl should point to the location of the local perl interpreter.
3. Non-windows users can set the \$windows flag to "thankfully no" in get_gfs.pl for more efficiency.

Simple Usage:

```
get_gfs.pl data DATE HR0 HR1 DHR VARS LEVS DIRECTORY
```

Note: some Windows setups will need to type:

```
perl get_gfs.pl data DATE HR0 HR1 DHR DIRECTORY
```

DATE = start time of the forecast YYYYMMDDHH
note: HH should be 00 06 12 or 18

HR0 = first forecast hour wanted

HR1 = last forecast hour wanted

DHR = forecast hour increment (forecast every 3, 6, 12, or 24 hours)

VARS = list of variables or "all"

ex. HGT:TMP:OZONE

ex. all

LEVS = list of levels, blanks replaced by an underscore, or "all"

ex. 500_mb:200_mb:surface

ex. all

DIRECTORY = directory in which to put the output

example: perl get_gfs.pl data 2006101800 0 12 6 UGRD:VGRD 200_mb .

example: perl get_gfs.pl data 2006101800 0 12 6 UGRD:VGRD 200_mb:500_mb:1000_mb .

example: perl get_gfs.pl data 2006101800 0 12 12 all surface .

regex metacharacters: () . ^ * [] \$ +

The get_gfs.pl script uses the perl regular expressions (regex) for string matching. Consequently the regex metacharacters should be quoted when they are part of the the search string. For example, trying to find

the following layer

```
"entire atmosphere (considered as a single_layer)"
"entire_atmosphere_(considered_as_a_single_layer)"
```

will not work because the parentheses are metacharacters. The following techniques will work.

Quoting the (and) characters

```
get_gfs.pl data 2012053000 0 6 3 TCDC "entire atmosphere \\\(considered as a single layer\\)" .
get_gfs.pl data 2012053000 0 6 3 TCDC entire_atmosphere_\\\\(considered_as_a_single_layer\\\\" .
```

Using a period (which matches all characters) to match the (and) characters

```
get_gfs.pl data 2012053000 0 6 3 TCDC "entire atmosphere .considered as a single layer." .
get_gfs.pl data 2012053000 0 6 3 TCDC entire_atmosphere_.considered_as_a_single_layer. .
```

How get_gfs.pl works

Get_gfs.pl is based on the get_inv.pl and get_grib.pl scripts. The advantage of get_gfs.pl is that the URL is built in as well as the looping over the forecast hours.

Metalanguage for get_gfs.pl data DATE HR0 HR1 DHR VARS LEVS DIRECTORY

```
# convert LEVS and VARS into REGEX
if (VARS == "all") {
    VARS=".";
}
else {
    VARS = substitute(VARS,':','|')
    VARS = substitute(VARS,'_',' ')
    VARS = ":(VARS)";
}

if (LEVS == "all") {
    LEVS=".";
}
LEVS = substitute(LEVS,':','|')
LEVS = substitute(LEVS,'_',' ')
LEVS = ":(LEVS)";
}

# loop over all forecast hours

for fhour = HR0, HR1, DHR
    URL= URL_name(DATE,fhour)
    URLinv= URL_name(DATE,fhour).idx

    inventory_array[] = get_inv(URLinv);
    for i = inventory..array[0] .. inventory_array[last]
        if (regex_match(LEVS,inventory_array[i]) and regex_match(VARS,inventory_array[i])) {
            add_to_curl_fetch_request(inventory_array[i]);
        }
    }
    curl_request(URL,curl_fetch_request,DIRECTORY);
endfor
```

Advanced Users

A user asked if it were possible to mix the variables and levels. For example, TMP @ 500 mb, HGT @ (250 and 700 mb). Of course you could run get-gfs.pl twice but that wouldn't be efficient.

It is possible because get-gfs.pl uses regular expressions and regular expressions are very powerful. All you need to remember is that get-gfs.pl converts the colon and underscore to a vertical bar and space, respectively for the VAR/LEV arguments.

Unix/Linux:

```
get-gfs.pl data 2006111500 0 12 12 all 'TMP.500 mb|HGT.(200 mb|700 mb)' data_dir
```

Windows:

```
get-gfs.pl data 2006111500 0 12 12 all "TMP.500 mb|HGT.(200 mb|700 mb)" C:\unix\
```

Other GRIB Data sets

One purpose of get_gfs.pl is to provide a simple script for downloading grib data using the partial httpd downloading protocol. The code was written so that it should be easily adapted to other grib+inv datasets.

Wrappers @ NCEP (NCO): get_data.sh

NCO (NCEP Central Operations) also has a wrapper, [get_data.sh](#).

Created: 10/2006, Updated: 5/2012
comments: Wesley.Ebisuzaki@noaa.gov

NOAA/ National Weather Service
National Centers for Environmental Prediction
Climate Prediction Center
5830 University Research Court
College Park, Maryland 20740
Climate Prediction Center Web Team
Page last modified: December 21, 2006

[Disclaimer](#)

[Privacy Notice](#)

[Privacy Policy](#)